

Programmation C II

TD/TP n° 1 : Trie dans un tableau 1D

Dr. KENMOGNE Edith Belise

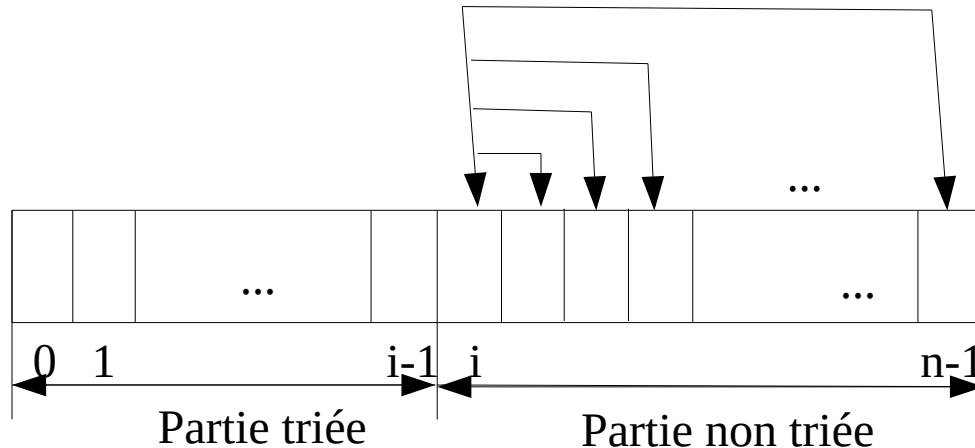
Exercice 1 : Le Tri par Sélection

Principe du tri par sélection : Appelé *selection sort* en anglais, c'est l'algorithme le plus simple qui soit. Il procède en $(n-1)$ étapes où n est la taille du tableau V à trier :

Etape 0 : Ecriture du plus petit élément de $V[0...n-1]$ dans $V[0]$.

Etape 1 : Ecriture du plus petit élément de $V[1...n-1]$ dans $V[1]$.

Etape i : Ecriture du plus petit élément de $V[i...n-1]$ dans $V[i]$.



Trie par sélection : Etape i

Questions :

a) Dérouler le principe du tri par sélection sur $[10, 8, 6, 2, 0]$ et sur $[0, 2, 6, 8, 10]$.

b) **Une version itérative du tri par sélection :** Ecrire une fonction qui prend en argument un tableau d'entiers v et sa taille et trie v par sélection. Testez votre fonction.

c) **Une version récursive du tri par sélection**

Une fonction est dite récursive si elle s'appelle. L'écriture d'une fonction récursive peut se faire en trois étapes : (1) On commence par traiter tous les cas pour lesquels il n'y a pas d'appel récursif. Ces cas constituent le seuil de coupure de la récursivité. Tous les appels récursifs doivent se ramener à ces cas. (2) On décrit en Français ou en Anglais le traitement à effectuer de manière récursive. (3) On traduit le traitement décrit à l'étape (2) en C.

c.1) Ecrire une version récursive du tri par sélection.

Indication : L'entête de cette fonction est `void tri_selection2(int *v, int i, int n)`. Ce qui signifie trier v sachant que $v[0...i-1]$ est trié et $v[i..n-1]$ non-trié. Le **seuil de coupure de la récursivité** est $(i=n-1)$. Remarquons que ; « Trier récursivement $v[0...n-1]$ par sélection sachant que $v[0...i-1]$ est trié et $v[i..n-1]$ non-trié revient (1) à rechercher l'indice (*ind*) du plus petit élément de $V[i...n-1]$, (2) l'échanger avec l'élément n°i de v et (3) à trier $v[0...n-1]$ sachant que $v[0...i]$ est trié et $v[i+1..n-1]$ non-trié ».

c.2) **Testez la version récursive. L'appel initial est `tri_selection2(v, 0, n)` car initialement la partie non-triée est $v[0...n-1]$.**

d) Soit $T(n)$ la complexité du tri par sélection sur un vecteur de taille n . On a $T(n)=T(n-1) + nc$, où c est une constante. Etablir que cette est de l'ordre de n^2 , notée $O(n^2)$. Calculer également la complexité en surface (espace mémoire).

e) Décrire le principe de tri par sélection basé sur la recherche du plus grand élément de la partie non-triée (qu'on précisera).

Exercice 2 : Le Tri par Insertion

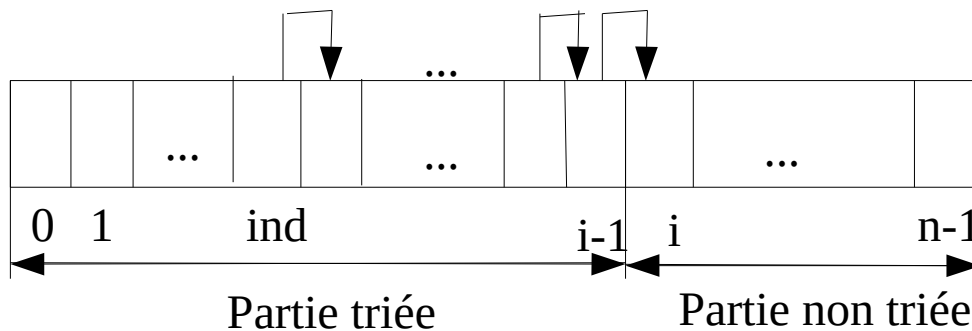
L'algorithme du tri par insertion est souvent assimilé au joueur qui trie ses cartes. Au début, le joueur a ses cartes placées en tas devant lui. Il les prend une par une de sa main droite pour les placer dans sa main gauche au bon endroit (c'est à dire en insérant systématiquement la dernière carte saisie à sa place).

Principe du tri par insertion : Il est appelé *selection sort* en anglais, Il procède en $(n-1)$ étapes où n est la taille du tableau V à trier :

Etape 1 : On suppose que $V[0...0]$ est trié. On insère l'élément $V[1]$ à sa bonne position dans $V[0...1]$.

Etape 2 : On suppose que $V[0...1]$ est trié. On insère l'élément $V[2]$ à sa bonne position dans $V[0...2]$.

Etape i : On suppose que $V[0...i-1]$ est trié. On insère l'élément $V[i]$ à sa bonne position dans $V[0...i]$. Pour ce faire, on peut effectuer des décalages. Après les décalages vers la droite on place $V[i]$ en position ind , où $V[ind]$ est le premier élément vers la gauche tel que $V[ind] \leq V[i]$.



Tri par insertion : Etape i.

Questions :

a) Dérouler le principe du tri par insertion sur $[10, 8, 6, 2, 0]$ et sur $[0, 2, 6, 8, 10]$.

b) **Une version itérative du tri par insertion :** Ecrire une fonction qui prend en argument un tableau d'entiers v et sa taille et trie v par insertion. Vous devez tester votre fonction.

c) **Une version récursive du tri par insertion**

c.1) Ecrire une version récursive du tri par insertion

Indication : L'entête de cette fonction est `void tri_insertion2(int *v, int i, int n)`. Ce qui signifie trier v sachant que $v[0...i-1]$ est trié et $v[i..n-1]$ non-trié. **Le seuil de coupure** de la recursivité est ($i=n$). Remarquons que ; « Trier récursivement $v[0...n-1]$ par insertion sachant que $v[0...i-1]$ est trié et $v[i..n-1]$ non-trié revient (1) à insérer $v[i]$ à sa bonne position et (2) à trier $v[0...n-1]$ sachant que $v[0...i]$ est trié et $v[i+1..n-1]$ non-trié ».

c.2) **Testez la version récursive.** L'appel initial est `tri_insertion2(v, 1, n)` car initialement la partie non-triée est $v[1...n-1]$.

d) Complexité du tri par insertion :

d.1) Calculer également la complexité en surface (espace mémoire).

d.2) Etablir que la complexité en temps (nombre d'opérations) du tri par insertion dans le meilleur des cas est de l'ordre de n , notée $O(n)$.

d.3) Etablir que la complexité en temps (nombre d'opérations) du tri par insertion dans le pire des cas est de l'ordre de n^2 , notée $O(n^2)$.

e) Décrire le principe de tri par insertion basé sur les décalages vers la gauche dans la partie triée (qu'on précisera).

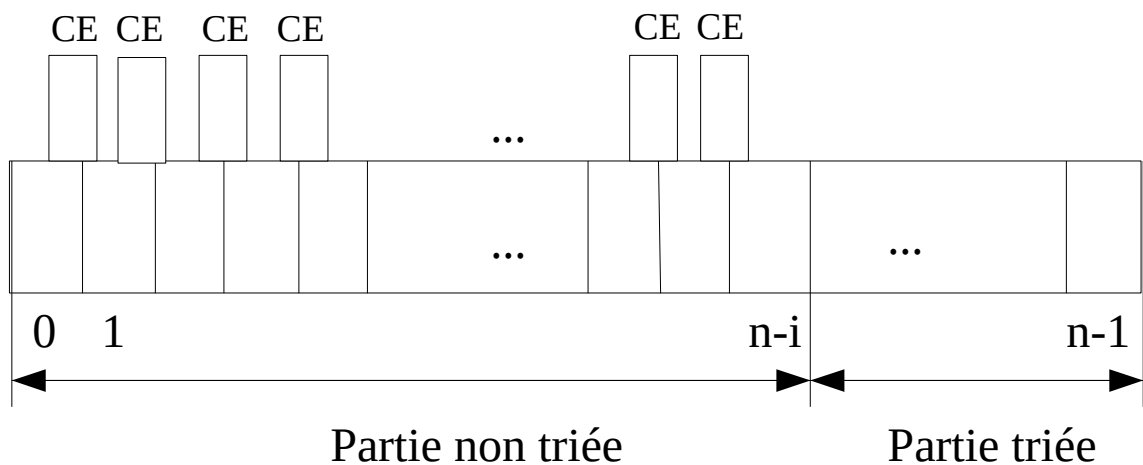
Exercice 3 : Le Tri à Bulles

Principe du trie bulle : Il est appelé *selection sort* en anglais, Il procède en $(n-1)$ étapes où n est la taille du tableau V à trier :

Etape 1 : On fait remonter 1^{er} plus grand élément à sa position définitive. Pour ce faire, on parcourt le tableau $V[0...n-1]$ de la gauche vers la droite en échangeant deux éléments consécutifs non ordonnés (ce qui revient à faire des comparaisons-échanges). Après le premier passage, le plus grand élément est situé à la fin du tableau ($V[n-1]$ contient le plus grand élément).

Etape 2 : On fait remonter 2^{ième} plus grand élément à sa position définitive. Pour ce faire, on parcourt le tableau $V[0...n-2]$ de la gauche vers la droite en échangeant deux éléments consécutifs non ordonnés (ce qui revient à faire des comparaisons-échanges). Après le deuxième passage, le 2^{ième} plus grand élément est dans $V[n-2]$.

Etape i : On fait remonter le $i^{\text{ième}}$ plus grand élément à sa position définitive un peu à la manière de bulles qu'on ferait remonter à la surface d'un liquide. D'où le nom de trie à bulles. Pour ce faire, on parcourt le tableau $V[0...n-i]$ de la gauche vers la droite en échangeant deux éléments consécutifs non ordonnés (comparaison-échange). Après le $i^{\text{ième}}$ passage, le $i^{\text{ième}}$ plus grand élément est dans $V[n-i]$.



Trie à bulles : Etape i. CE signifie Comparaison-Echange

Questions :

a) Dérouler le principe du trie bulles sur $[10, 8, 6, 2, 0]$ et sur $[0, 2, 6, 8, 10]$.

b) **Une version itérative du trie à bulles :** Ecrire une fonction qui prend en argument un tableau d'entiers v et sa taille et trie v . Vous devez tester votre fonction.

c) **Une version récursive du trie à bulles :**

c.1) Ecrire une version récursive du trie à bulles

Indication : L'entête de cette fonction peut être `void tri_bulles2(int *v, int i, int n)`. Ce qui signifie trier v sachant que $v[0...n-i]$ est non-trié et $v[n-i+1...n-1]$ est trié. Le seul cas pour lequel il n'y a pas d'appel récursif est le cas où $(i=n)$. Remarquons que ; « Trier récursivement $v[0...n-1]$ par bulles sachant que $v[0...n-i]$ est non trié et $v[n-i+1...n-1]$ trié revient (1) à faire remonter le $(i+1)^{\text{ième}}$ plus grand élément en sa position définitive et (2) à trier $v[0...n-1]$ sachant que $v[0...n-i-1]$ est non-trié et $v[n-i...n-1]$ trié ».

c.2) Testez la version récursive. L'appel initial est `tri_bulles(v, 1, n)` car initialement on recherche le 1^{er} plus grand (la partie non-triée est $v[0...n-1]$).

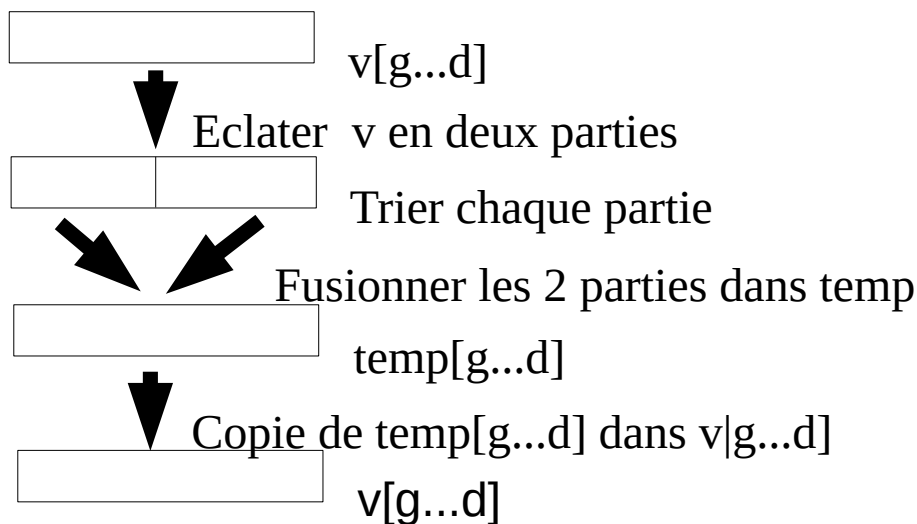
d) Soit $T(n)$ la complexité du trie à bulles sur un vecteur de taille n . On a $T(n)=T(n-1) + nc$, où c est une constante. Etablir que cette complexité est de l'ordre de n^2 , notée $O(n^2)$. Calculer également la complexité en surface (espace mémoire).

e) On peut améliorer le tri à bulles en remarquant que si lors d'un passage on n'effectue pas d'échange, alors le vecteur est trié.

- e.1)** Etablir que la complexité en temps (nombre d'opérations) du trie par insertion dans le meilleur des cas est de l'ordre de n , notée $O(n)$.
- e.2)** Etablir que la complexité en temps (nombre d'opérations) du trie par insertion dans le pire des cas est de l'ordre de n^2 , notée $O(n^2)$.
- e.3)** re-écrire une fonction itérative qui effectue le trie à bulles.
- f)** Décrire le principe de trie à bulles basé sur des comparaisons-échanges de la droite vers la gauche.

Exercice 4 : Le Tri par fusion

Principe du trie par fusion : Appelée *merge sort*, cette méthode adopte une approche « *diviser pour régner* » : on partage le tableau en deux parties de taille $n/2$ que l'on trie par un appel récursif, puis on fusionne les deux parties triées. Malheureusement, une difficulté se présente : Il n'est pas possible d'avoir le résultat de la fusion de deux demi-tableaux de v dans v sans tableau intermédiaire. C'est pourquoi nous allons utiliser un tableau provisoire *temp* pour y stocker le résultat de la fusion. Il faudrait aussi prévoir un seuil permettant de couper la récursivité. Ainsi nous pouvons admettre qu'il n'y a pas d'appel récursif si la taille du vecteur est inférieure à TAILLE_MIN=5. Dans ce cas on utilise effectivement un trie itératif : trie par sélection, trie par insertion ou trie à bulles.



Trie par fusion : Les étapes

Questions :

a) Copie de vecteur : Ecrire une fonction `void copie(int *v, int g, int d, int *temp)` qui copie $\text{temp}[g\dots d]$ dans $v[g\dots d]$.

b) Trie par insertion : Ecrire une fonction itérative `void trie_insertion2(int *v, int g, int d)` qui trie $v[g\dots d]$ par ordre croissant.

c) Fusion de deux sous-tableaux

c.1) Fusion de deux tableaux avec trois boucles

c.1.a) Principe de fusion (à trois boucles) de $a[ga\dots da]$ et $b[gb\dots db]$ dans $c[ga\dots db]$, $gb==da+1$: On considère un index ia de a initialisé à ga , un index ib de b initialisé à gb et un index ic de c initialisé à ga . Tant que aucun des deux tableaux a et b n'est épuisé, à chaque étape, la plus petite valeur indexée par ia et ib est ajoutée à c à la position ic puis ic et l'index ayant produit la plus petite valeur sont incrémentés de 1. Dès qu'un des deux vecteurs a et b est épuisé ($ia==da+1$ ou $ib==db+1$), on recopie le reste des éléments du vecteur non épuisé (à partir de son index) dans c (à partir de son index de ic).

c.1.b) Dérouler ce principe pour fusionner $[1, 3, 5, 7, 9]$ et $[2, 4, 6, 8, 10]$.

c.1.c) `void fusion1(int *c, int *a, int ga, int da, int *b, int gb, int db)` : Cette fonction prend en argument, (1) un vecteur trié par ordre croissant $a[ga\dots da]$ et (2) un deuxième vecteur trié par ordre croissant $b[gb\dots db]$ tels $gb=da+1$ et les fusionne dans un vecteur $c[ga\dots db]$. Ecrire et tester cette fonction.

c.2) Fusion de deux sous-tableaux avec une seule boucle

c.2.a) Principe de fusion (à une boucle) de $a[ga...da]$ et $b[gb...db]$ dans $c[ga...db]$, $gb==da+1$: On considère un index ia de a initialisé à ga , un index ib de b initialisé à gb et un index ic de c initialisé à ga . Tant que un des deux tableaux a et b n'est pas épuisé, à chaque étape, **(1) si** a est épuisé ($ia==da+1$), on copie $a[ia]$ dans $c[ic]$ puis on incrémente ia et ic de 1. **(2) sinon si** b est épuisé ($ib==db+1$), on copie $b[ib]$ dans $c[ic]$ puis on incrémente ib et ic de 1, **(3) sinon si** $a[ia]<b[ib]$, on copie $a[ia]$ dans $c[ic]$ puis on incrémente ia et ic de 1, **(4) sinon** (on a $a[ia]>=b[ib]$) on copie $b[ib]$ dans $c[ic]$ puis on incrémente ib et ic de 1.

c.2.b) Dérouler ce principe pour fusionner $[1, 3, 5, 7, 9]$ et $[2, 4, 6, 8, 10]$.

c.2.c) `void fusion1(int *c, int *a, int ga, int da, int *b, int gb, int db)` : Cette fonction prend en argument, (1) un vecteur trié par ordre croissant $a[ga...da]$ et (2) un deuxième vecteur trié par ordre croissant $b[gb...db]$ tels $gb=da+1$ et les fusionne dans un vecteur $c[ga...db]$. Ecrire et tester cette fonction.

d) Trie par fusion

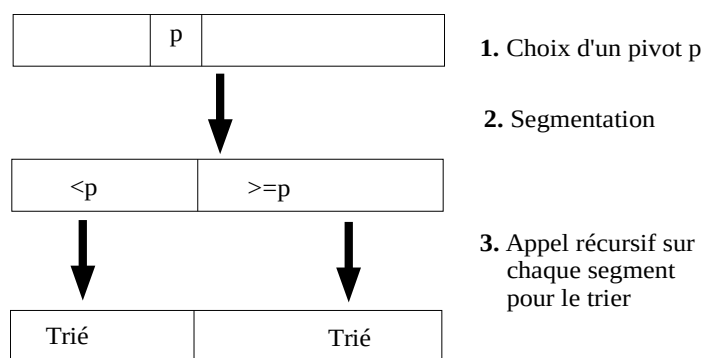
d.1) `void trie_fusion(int *v, int gv, int dv)` : Cette fonction trie par fusion $v[gv...dv]$. Ecrire cette fonction.

d.2) Tester cette fonction. L'appel initial est `trie_fusion(v, 0, n-1)`.

c) Soit $T(n)$ la complexité du trie par fusion sur un vecteur de taille n . On a $T(n) = 2T(n/2) + cn$, où c est une constante. Etablir que cette complexité est de l'ordre de $n \log(n)$, notée $O(n \log(n))$. Calculer également la complexité en surface (espace mémoire).

Exercice 5 : Le Tri rapide

Principe du trie rapide : Appelé *quick sort* en anglais, ce tri adopte lui aussi une démarche de type « diviser pour régner » de manière à éviter la fusion: on commence par segmenter le tableau autour d'un pivot choisi parmi les éléments du tableau en plaçant les éléments qui lui sont inférieurs à sa gauche, et les éléments qui lui sont supérieurs, à sa droite. Ensuite, les parties gauche et droite sont triées par l'intermédiaire d'un appel récursif. **Il faudrait aussi prévoir un seuil permettant de couper la récursivité.** Ainsi nous pouvons admettre qu'il n'y a pas d'appel récursif si la taille du vecteur est inférieure ou égale à `TAILLE_MIN=5`. Dans ce cas on effectue un trie itératif : *trie par sélection*, *trie par insertion* ou *trie à bulles*.



Trie rapide : Les étapes

Questions :

a) Trie par insertion : Ecrire une fonction itérative `void trie_insertion2(int *v, int g, int d)` qui trie $v[g...d]$ par ordre croissant.

b) Recherche d'un pivot et segmentation

Pivot : `int search_pivot(int *v, int g, int d)` : Recherche un pivot de $v[g...d]$ et retourne son index : si la valeur retournée est `index` alors la valeur du pivot est $v[index]$. La Post-condition de la fonction `search_pivot` est que le vecteur v contient au moins trois valeurs distinctes. Ecrire la fonction `search_pivot`.

Le pivot doit être choisi de manière à diviser le vecteur v en deux parties non vides. Par exemple, en choisissant 1 comme pivot, on ne peut pas décomposer le vecteur {1, 2, 3, 20, 25, 15, 5, 7, 40, 45} en deux parties non-vides comme dans la figure précédente. Dans ce cas précis, le sous-vecteur de gauche est vide.

La recherche d'un pivot peut se faire en deux étapes. **(1)** Dans la première étape, on recherche trois indices i, j et k tels que les trois valeurs $v[i], v[j]$ et $v[k]$ soient distinctes. Pour ce faire on initialise i à g et on recherche le premier indice j tel que $v[i]$ soit distinct de $v[j]$: **$i=g ; j=g+1 ; \text{tant que } (v[i]==v[j]) \text{ } j++$** . Ensuite on initialise k à $j+1$ et on recherche le premier k distinct tel que $v[k]$ soit distinct de $v[i]$ et de $v[j]$: **$k=j+1 ; \text{tant que } ((v[i]==v[k]) \text{ ou } (v[j]==v[k])) \text{ } k++$** . **(2)** Dans la deuxième étape, on recherche la valeur intermédiaire des trois valeurs $v[i], v[j]$ et $v[k]$ puis on retourne son indice. Par exemple si $v[i]=1, v[j]=2$ et $v[k]=3$, la valeur intermédiaire est $v[j]$ et on retourne l'indice j .

Segmentation : *int segmentation(int *v, int g, int d)* : Cette fonction appelle `search_pivot` pour obtenir un pivot, puis utilise ce pivot pour segmenter de $v[g...d]$ en deux sous-vecteurs (le sous-vecteur de gauche et le sous-vecteur de droite) puis retourne l'index le plus à droite du sous-vecteur de gauche. Ecrire cette fonction.

La segmentation peut se faire en considérant deux tamis représentés par deux index tg (tamis de gauche) et td (tamis de droite). Le premier tamis **tg est initialisé à g** et le deuxième tamis **td est initialisé à d** . Le premier tamis tg , se déplace de gauche à droite, laisse passer uniquement les valeurs strictement inférieures au pivot et se bloque sur une valeur supérieure ou égale au pivot (c'est à dire si $V[tg] \geq \text{pivot}$) : **tant que $(V[tg] < \text{pivot})$, incrémenter tg de 1**. Le deuxième tamis td se déplace de droite à gauche, laisse passer uniquement les valeurs supérieures ou égales au pivot et se bloque sur une valeur strictement inférieure au pivot : **tant que $(V[td] > \text{pivot})$, décrémenter td de 1**. **Quand les deux tamis sont bloqués, on les débloque uniquement dans le cas où l'inégalité $(tg \leq td)$ est vraie en permutant les valeurs de $V[tg]$ et de $V[td]$. L'algorithme s'arrête dès que les deux tamis se croisent, c'est à dire dès que l'inégalité $(tg \leq td)$ est fausse. L'index le plus à droite du sous-vecteur de gauche est $(tg-1)$. C'est la valeur retournée par la fonction `segmentation`.**

Illustrer le déroulement de la segmentation sur un exemple: {1, 2, 3, 20, 25, 15, 5, 7, 40, 45} avec 15 comme pivot.

c) Trie rapide

c.1) *void trie_rapide(int *v, int gv, int dv)* : Cette fonction trie $v[gv...dv]$. Ecrire cette fonction.

c.2) Tester cette fonction. L'appel initial est `trie_rapide(v, 0, n-1)`.

d) Complexité du trie rapide

d.1) Calculer également la complexité en surface (espace mémoire).

d.2) Etablir que la complexité en temps (nombre d'opérations) du trie rapide dans le pire des cas est de l'ordre de n^2 , notée $O(n^2)$.

d.3) Soit $T(n)$ la complexité moyenne (la segmentation produit toujours deux parties de même taille) du trie rapide sur un vecteur de taille n . On a $T(n) = 2T(n/2) + cn$, où c est une constante. Etablir que cette complexité est de l'ordre de $n \log(n)$, notée $O(n \log(n))$.

Exercice 6 : Le Tri par dénombrement (encore appelé trie par comptage de fréquence)

1) Condition d'application du trie par dénombrement :

Le tri par dénombrement est applicable lorsque le nombre de valeurs distincts est largement inférieur à la taille du vecteur. Dans le cas de $V=\{1, 2, 0, 0, 1, 1, 1, 2, 2, 0\}$, on a trois (3) valeur distinctes et la taille de V est dix (10).

2) Principe de base du trie par dénombrement : Les valeurs de V sont prises dans l'intervalle $[0, k]$

Dans ce cas, le tri par dénombrement du vecteur V peut se faire en deux étapes :

Etape 0 : Calculer la fréquence de chaque valeur x de V dans $f[x]$ où f est un tableau de taille k .

Etape 1 : **(1)** Ecrire $f[x]$ occurrences de 0 dans V à partir de la position 0. **(2)**

(2) Pour x de 1 à k écrire $f[x]$ occurrences de x dans V à partir de l'index $f[0] + f[1] + \dots + f[x-1]$

Questions :

2.1) Dérouler le principe du trie par dénombrement sur $V=\{1, 2, 0, 0, 1, 1, 1, 2, 2, 0\}$.

2.2) Dans le cas où $V=\{1000, 2000, 0, 0, 1000, 1000, 1000, 2000, 2000, 0\}$, donner (1) la valeur de k , (2) la taille du vecteur f et (3) le nombre d'exécution du corps de la boucle pour.

2.3) En déduire que l'application de ce principe de base coûte trop cher, en nombres d'opérations et en espace mémoire, dans le cas où les valeurs sont dispersées (ou trop distantes).

3) Principe général du trie par dénombrement

Dans ce cas, le trie par dénombrement du vecteur V peut se faire en quatre étapes :

Etape 0 : Ecrire dans le tableau R toutes les valeurs distinctes de V dans l'ordre croissant.

Etape 1 : Remplacer dans V chaque valeur x par son rang (c'est à dire ind tel que $R[ind]$ est égal à x).

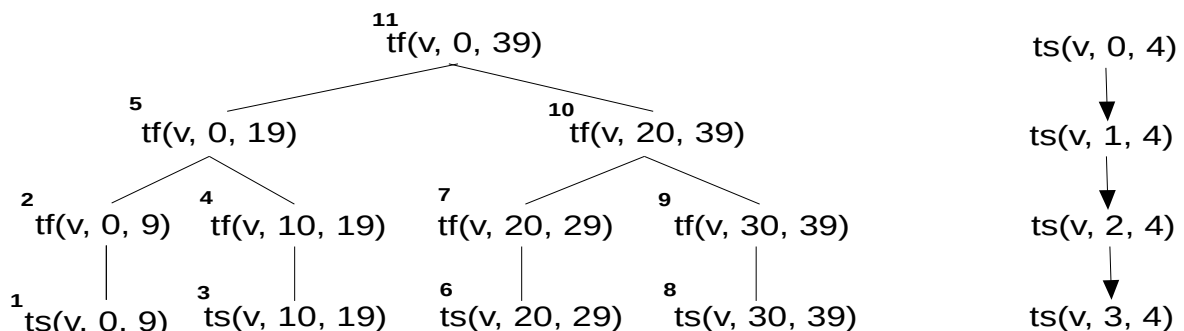
Etape 2 : **Appliquer le principe de base du trie par dénombrement à V .**

Etape 3 : Remplacer dans V chaque valeur ind par $R[ind]$.

Dérouler le principe général du trie par dénombrement sur $V=\{1000, 2000, 0, 0, 1000, 1000, 1000, 2000, 2000, 0\}$.

Retour sur la Récursivité

La figure suivante illustre l'arborescence des appels récursifs du trie par fusion et de la version récursive du trie par sélection. Pour le trie par fusion, la taille de coupure de la récursivité est (TAILLE_MIN) 11. Les feuilles des deux arbres représentent les noeuds de coupure de la récursivité. Les quatre 0feuilles de l'arbre de gauche sont : $ts(v, 0, 9)$, $ts(v, 10, 19)$, $ts(v, 20, 29)$ et $ts(v, 30, 39)$. L'unique feuille de l'arbre de droite est : $ts(v, 3, 4)$. Les numéros donnent l'ordre de finalisation des appels encore appelé l'ordre de déroulement du trie par fusion : **Le traitement d'un noeud père (tâche père) se termine après les traitements de ses noeuds fils (tâches fils).**



ts=Trie par Sélection, tf=Trie par Fusion, ts=Trie par Sélection

**Arborescence des appels récursifs
du trie par fusion et du trie par sélection**